

3.2 Resumen Estructuras Repetitivas

Emilio Rojas Badillo

Instituto Tecnológico de Querétaro

Fundamentos de Programación

María Luisa Montes Almanza

2025-10-08

Índice

1. Introducción	3
1.1. Definición y Control General	3
2. Variables de Control de Ciclos	4
3. Tipos de Estructuras Repetitivas	5
3.1. Estructura de Control Repetitiva Mientras (while)	5
3.2. Estructura de Control Repetitiva Hacer_mientras (do while)	5
3.3. Estructura de Control Repetitiva Desde (for)	6
4. Ciclos Anidados	7
5. Referencias	8

1. Introducción

Las estructuras repetitivas, también conocidas como estructuras de control de iteración condicional, constituyen uno de los pilares de la programación estructurada, junto con las estructuras secuenciales y selectivas. Los siguientes detalles se basan en el enfoque presentado en Diseño de algoritmos y su codificación en lenguaje C por María Adriana Corona Nakamura y María de los Ángeles Ancona Valdez. El texto de Nakamura et al. es una obra fundamental que introduce los conceptos básicos de la programación, haciendo énfasis en el diseño de algoritmos y la codificación en lenguaje C. La metodología expuesta sigue las reglas clásicas del diseño de programas estructurados.

1.1. Definición y Control General

La estructura de control repetitiva se caracteriza por la repetición de una acción (una o varias instrucciones) siempre y cuando se cumpla una determinada condición. A estos segmentos de código se les llama bucles, ciclos o iteraciones. Condición de Terminación Para que el ciclo pueda finalizar, la acción o bloque de instrucciones ejecutado debe modificar la(s) variable(s) de control que intervienen en la condición. Si esta condición no se vuelve falsa (cero), el bucle se repetirá indefinidamente.

2. Variables de Control de Ciclos

Para administrar los bucles, se emplean variables especializadas:

- **Contadores:** Variables cuyo valor se incrementa o decrementa en una cantidad constante cada vez que ocurre un suceso. Controlan o determinan la cantidad de veces que se repite un proceso. Deben inicializarse antes de entrar al ciclo.
- **Acumuladores:** Funcionan de manera similar a los contadores, pero su incremento o decremento es variable. Se utilizan para sumar o restar un conjunto de valores sucesivos, y generalmente se inicializan en 0 (para sumas o restas) o en 1 (para productos o multiplicaciones).
- **Centinelas:** Variables que, al cambiar su valor dentro del bucle, indican el fin del ciclo. La repetición controlada por un centinela se considera una repetición indefinida, ya que el número de repeticiones es desconocido de antemano por el programa.
- **Banderas:** Variables que solo pueden tomar dos valores opuestos, habitualmente 1 (verdadero) o 0 (falso). Son muy utilizadas en las búsquedas.

3. Tipos de Estructuras Repetitivas

Existen tres estructuras repetitivas principales:

3.1. Estructura de Control Repetitiva Mientras (**while**)

- Evaluación de la Condición: La condición (expresión lógica) se evalúa antes del ciclo.
- Ejecución: Si la condición es verdadera (distinto de cero), el bloque de instrucciones se ejecuta. Si la condición es falsa cuando se revisa por primera vez, el bucle no se ejecuta nunca.
- Comportamiento: Mientras la condición sea verdadera, el bloque de instrucciones se ejecuta indefinidamente, por lo que debe existir al menos una instrucción dentro del bloque que modifique la condición para asegurar la finalización.
- Sintaxis en C: No se debe escribir punto y coma después de la condición while (condición) en Lenguaje C para evitar que el ciclo no se realice. Si se requiere más de una instrucción, estas se agrupan con inicio-fin en pseudocódigo o con {} en C.

3.2. Estructura de Control Repetitiva Hacer_mientras (**do while**)

- Ejecución Garantizada: A diferencia del mientras, esta estructura siempre entra al ciclo por lo menos una vez, ya que la condición se evalúa al final.
- Evaluación de la Condición: La condición se revisa después de ejecutar el cuerpo del bucle.
- Repetición: Si la condición resulta verdadera, se vuelve a entrar al ciclo y el proceso se repite hasta que la condición se vuelve falsa, momento en el que se sale del bucle.
- Obligación de Bloque: Si la estructura tiene más de una instrucción, necesita obligadamente las palabras reservadas inicio-fin (o {})
- Sintaxis en C: A diferencia del while simple, al final del do_while sí lleva punto y coma después de la instrucción while (condición).

3.3. Estructura de Control Repetitiva Desde (for)

- **Uso Principal:** Es considerada la estructura repetitiva más utilizada y simple de manejar, empleada cuando se conoce de antemano el número determinado de veces que se repetirán las instrucciones. Es fundamental para manejar arreglos.
- **Control de Variables:** Se debe conocer de antemano el valor de la variable inicial y final antes de entrar al ciclo.
- **Componentes:** En pseudocódigo y Lenguaje C, el for incluye:
 1. **Expresión de Inicialización (`expr_ini(s)`):** Establece el valor inicial de la(s) variable(s) de control.
 2. **Condición (`cond`):** Expresión booleana que determina cuándo finalizará el ciclo.
 3. **Expresión de Incremento/Decremento (`inc(s)`):** Define cómo cambiará la variable de control en cada repetición.
- **Proceso de Ejecución:** Primero se ejecuta la inicialización, luego se evalúa la condición. Si es verdadera, se ejecuta el bloque de instrucciones, se realiza el incremento/decremento (el cual es automático), y se vuelve a verificar la condición.
- **Omisión de Expresiones:** Es posible omitir cualquiera de los tres componentes (inicialización, condición, incremento), pero los separadores (punto y coma en C) deben permanecer. Si la condición se omite, se asume un valor verdadero (1), lo que resulta en un bucle infinito.

4. Ciclos Anidados

Los algoritmos pueden contener múltiples bucles, que pueden ser independientes o anidados.

La iteración anidada consiste en incluir una instrucción de repetición (`while`, `do-while` o `for`)

dentro del cuerpo de otra.

- Ejecución: Por cada iteración del ciclo exterior, el ciclo interior completa su grupo total de iteraciones.
- Ejemplo: Si un ciclo exterior se ejecuta 2 veces y uno interior 3 veces, la instrucción más interna se ejecutará $2 \text{ veces } 3 = 6 \text{ veces}$. Este concepto de anidamiento es también utilizado en estructuras selectivas (como `if` anidados).

5. Referencias

Nakamura, M. A. C., & Ancona Valdez, M. de los Á. (2011). Diseño de algoritmos y su codificación en lenguaje C. McGraw-Hill/Interamericana Editores, S.A. de C.V..